SEES Reference and Known-Truth Validation Tutorial

Sieve Estimation of Economic Structural Models

econirl package documentation

May 2026

RTD page: https://econirl.readthedocs.io/en/latest/estimators/sees.html **Source:** https://github.com/rawatpranjal/EconIRL/tree/main

1 Purpose

SEES estimates structural dynamic discrete choice models by approximating the integrated value function with deterministic sieve basis functions. NFXP solves the Bellman fixed point inside every likelihood evaluation. MPEC treats the value function as a free optimizer vector and imposes the Bellman equation as an equality constraint. SEES keeps the joint-optimization idea but replaces the full value vector $V \in \mathbb{R}^{|\mathcal{S}|}$ with

$$V_K(s;\alpha) = \psi_K(s)^\top \alpha,$$

and penalizes Bellman residuals instead of imposing exact equality.

This document is a known-truth validation tutorial. It does not use real data. The synthetic DGP supplies the true reward, transition law, value function, policy, and Type A/B/C counterfactual oracle objects. The validation question is whether the SEES implementation recovers these known objects on a medium-scale finite panel while enforcing the Bellman equation tightly enough for structural use.

2 Source Paper

The source paper is Luo and Sang (2024). The paper proposes a sieve-based efficient estimator for structural dynamic models. The value function is approximated by basis functions, and the equilibrium condition is imposed through a penalty. The estimator jointly optimizes structural parameters and sieve coefficients:

$$(\hat{\theta}, \hat{\alpha}) = \arg\max_{\theta,\alpha} \ell_n(\theta,\alpha) - \omega_n \left\| V_K(\alpha) - T_\theta V_K(\alpha) \right\|^2.$$

The theoretical target is not a rough simulator fit. Under smoothness, identification, sieve-approximation, and penalty-rate conditions, the structural parameter estimator is consistent, asymptotically normal, and efficient. The finite discrete-state limiting case is important for this package: when the sieve has enough columns to span the finite-state value vector and the penalty is large, SEES approaches the MPEC formulation.

The paper's main warning is also practical. SEES needs a basis rich enough to represent the value function and a penalty large enough to enforce equilibrium. Too few basis functions create projection bias. Too small a penalty allows the likelihood to fit choices with a value approximation that does not solve the dynamic model.

3 When to Use SEES

Use SEES when the model is structural, the reward is parametric, transitions are known or first-stage estimated, and the state space is large enough that NFXP or MPEC are costly but still smooth enough for deterministic basis functions to approximate V. SEES is a structural approximation method, not a behavioral cloning method.

Do not use SEES as a black-box rescue for a non-identified DGP. If the reward features are state-only and identical across actions, if an action is missing, or if the exit/scale normalization is absent, the likelihood cannot identify the structural parameters. The known-truth gates below are intentionally hard: a non-smoke run must pass Bellman residual, standard-error, reward, value, policy, and counterfactual checks.

4 Model

The observed data are state-action-next-state trajectories

$$\{(s_{it}, a_{it}, s_{i,t+1}) : i = 1, \ldots, N,\ t = 1, \ldots, T\}.$$

Flow payoff is linear in known features:

$$u_\theta(s, a) = \phi(s, a)^\top \theta.$$

Transitions are

$$P_a(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a),$$

the discount factor is β, and the type-I extreme-value shock scale is σ. Given V, the choice-specific value is

$$Q_\theta(s, a; V) = u_\theta(s, a) + \beta \sum_{s'} P_a(s, s') V(s').$$

The logit policy implied by (θ, V) is

$$\pi_{\theta,V}(a \mid s) = \frac{\exp(Q_\theta(s, a; V)/\sigma)}{\sum_b \exp(Q_\theta(s, b; V)/\sigma)}.$$

The soft Bellman operator is

$$T_\theta V(s) = \sigma \log \sum_a \exp\left(\frac{u_\theta(s, a) + \beta \sum_{s'} P_a(s, s') V(s')}{\sigma}\right).$$

5 SEES Estimator

Let $\Psi \in \mathbb{R}^{|\mathcal{S}| \times K}$ be the sieve basis matrix and $V_K = \Psi\alpha$. The implementation supports B-splines, polynomials, and Fourier bases. The known-truth validation uses B-splines because Luo and Sang use spline sieves as the paper's default smooth basis.

The penalized criterion is

$$\mathcal{Q}_n(\theta, \alpha) = \sum_{i,t} \log \pi_{\theta,\Psi\alpha}(a_{it} \mid s_{it}) - \omega \sum_s \left[\Psi_s \alpha - T_\theta(\Psi\alpha)(s)\right]^2.$$

The optimizer variable is $x = (\theta, \alpha)$. There is no nested value iteration. The package uses JAX automatic differentiation and L-BFGS-B over the joint vector. The reported log likelihood is the unpenalized choice log likelihood at the final $(\hat{\theta}, \hat{\alpha})$; the Bellman violation is reported separately and is part of the hard validation gate.

5.1 Standard Errors

Let the Hessian of the penalized objective in (θ, α) blocks be

$$H = \begin{pmatrix} H_{\theta\theta} & H_{\theta\alpha} \\ H_{\alpha\theta} & H_{\alpha\alpha} \end{pmatrix}.$$

SEES treats α as a nuisance approximation coefficient. The marginal information for θ uses the Schur complement:

$$\tilde{H}_\theta = H_{\theta\theta} - H_{\theta\alpha} H_{\alpha\alpha}^{-1} H_{\alpha\theta}.$$

The validation run computes this marginal Hessian and requires finite standard errors. A run with good point estimates but singular standard errors is not treated as a completed structural validation.

6 Identification Checks

1. **Reward rank.** The action-dependent feature matrix must have full column rank after dropping zero rows.
2. **Action support.** Each action must appear in the finite panel so the likelihood can discipline payoff differences.
3. **State coverage.** The canonical tabular validation observes every state in the regular state space.
4. **Normalization.** The reward scale is fixed at $\sigma = 1$, and the synthetic DGP uses an exit action and absorbing state as a zero-reward anchor.
5. **Basis adequacy.** The sieve must be rich enough for the DGP. In the finite-state validation, $K = |\mathcal{S}| = 21$, so the B-spline basis has enough columns to span a 21-state value vector.
6. **Penalty strength.** The Bellman residual is not cosmetic. The non-smoke validation gates on the final residual and fails if the penalty allows V_K to drift away from $T_\theta V_K$.

7 Known-Truth DGP

The canonical SEES validation cell is `canonical_low_action`. It is the same low-dimensional action-dependent structural benchmark used for NFXP, CCP, and MPEC.

Table 1: Canonical SEES DGP.

Quantity	Value
Regular states	20
Absorbing states	1
Total states	21
Actions	3
Exit action	2
Discount factor	0.95
Shock scale	1.0
Reward mode	action-dependent
Reward dimension	low
State mode	low-dimensional
Simulated individuals	2,000
Periods per individual	80
Observations	160,000

The action-dependent features are

$$\phi(s,a) = \begin{cases} (1, x_s, 0, 0) & a = 0, \\ (0, 0, 1, x_s) & a = 1, \\ (0, 0, 0, 0) & a = 2. \end{cases}$$

The exit action has zero reward and sends the process to the absorbing state. The true parameter vector is:

Table 2: True reward parameters.

Parameter	Truth
`action_0_intercept`	0.10
`action_0_progress`	0.50
`action_1_intercept`	0.00
`action_1_progress`	-0.20

8 Pre-Estimation Diagnostics

Run diagnostics before fitting. For SEES, basis and penalty choices are only meaningful after identification checks pass.

```
from experiments.known_truth import (
    SimulationConfig,
    build_known_truth_dgp,
    get_cell,
    run_pre_estimation_diagnostics,
    simulate_known_truth_panel,
)

cell = get_cell("canonical_low_action")
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(
    dgp,
    SimulationConfig(n_individuals=2000, n_periods=80, seed=42),
)
diagnostics = run_pre_estimation_diagnostics(dgp, panel)

assert diagnostics.passed
assert diagnostics.feature_rank == diagnostics.num_features
assert diagnostics.is_action_dependent
assert diagnostics.state_action_coverage == 1.0
```

9 Run the Tutorial

From the repository root:

```
PYTHONPATH=src:. python -m experiments.known_truth \
    --estimator SEES \
    --cell-id canonical_low_action \
    --output-dir outputs/known_truth \
    --show-progress \
    --verbose
```

The command builds the DGP, simulates the panel with `tqdm` progress, runs pre-estimation diagnostics, fits SEES with Schur-complement standard errors, evaluates reward/value/Q/policy recovery, solves Type A/B/C counterfactual oracles, writes one `result.json`, and raises if any non-smoke hard gate fails.

Direct estimator usage:

```
from experiments.known_truth import (
    build_known_truth_dgp,
    get_cell,
    simulate_known_truth_panel,
)
from econirl.estimation.sees import SEESEstimator

cell = get_cell("canonical_low_action")
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(dgp, cell.simulation_config)

estimator = SEESEstimator(
    basis_type="bspline",
    basis_dim=dgp.problem.num_states,
    penalty_weight=100.0,
    max_iter=1000,
```

```
    tol=1e-7,
    compute_se=True,
    verbose=True,
)

summary = estimator.estimate(
    panel=panel,
    utility=dgp.utility(),
    problem=dgp.problem,
    transitions=dgp.transitions,
)
```

10 Generated Validation Results

This section is generated by `sees_run.py` from the `experiments.known_truth` harness and the canonical known-truth DGP.

10.1 Pre-Estimation Checks

Table 3: Pre-estimation checks from the known-truth SEES run.

Check	Value	Status
Feature rank	4 / 4	pass
Feature condition number	4.512	pass
Transition row error	2.42×10^{-8}	pass
Observed states	21 / 21	pass
State-action coverage	1.000	pass
Action shares	0.345, 0.330, 0.325	pass
Minimum action share	0.325	pass
Exit/absorbing anchor	true	pass

10.2 Estimator Fit

The medium-scale validation run uses 2,000 individuals, 80 periods per individual, and 160,000 observations.

Table 4: Known-truth SEES run summary.

Quantity	Value
Optimizer absolute-gradient flag	false
L-BFGS-B iterations	135
Log-likelihood	-174872.5938
Estimation time	10.06 seconds
Basis type	bspline
Basis dimension	21
Penalty weight	100.0
Bellman violation	0.044579
Standard errors finite	true

The optimizer flag is reported exactly as returned by the L-BFGS-B wrapper. The hard validation gate for SEES is not that sample-summed absolute-gradient flag; it is the structural Bellman residual, finite standard errors, and known-truth recovery metrics reported below.

Table 5: Parameter recovery.

Parameter	Truth	Estimate	SE	Error
`action_0_intercept`	0.100000	0.069182	0.125380	-0.030818
`action_0_progress`	0.500000	0.554673	0.077976	0.054673
`action_1_intercept`	0.000000	-0.023757	0.129408	-0.023757
`action_1_progress`	-0.200000	-0.186550	0.078555	0.013450

Table 6: Recovery metrics.

Metric	Value
Parameter RMSE	0.034220
Parameter relative RMSE	0.124955
Parameter cosine similarity	0.995485
Reward RMSE	0.013579
Value RMSE	0.080710
Q RMSE	0.080546
Policy KL	2.62×10^{-4}
Policy total variation	0.008884
Policy max state L1	0.038960

Table 7: Hard recovery gates.

Gate	Threshold	Value	Status
bellman_violation	≤ 0.050000	0.044579	pass
standard_errors_finite	true	true	pass
parameter_cosine	≥ 0.990000	0.995485	pass
parameter_relative_rmse	≤ 0.150000	0.124955	pass
reward_rmse	≤ 0.030000	0.013579	pass
policy_tv	≤ 0.020000	0.008884	pass
value_rmse	≤ 0.100000	0.080710	pass
q_rmse	≤ 0.100000	0.080546	pass
type_a_regret	≤ 0.010000	0.000526	pass
type_b_regret	≤ 0.010000	0.001281	pass
type_c_regret	≤ 0.010000	0.000313	pass

10.3 Counterfactual Recovery

Table 8: Counterfactual recovery against known oracle objects.

Counterfactual	Policy TV	Policy KL	Value RMSE	Regret
Type A	0.008031	1.85×10^{-4}	0.000621	0.000526
Type B	0.008981	2.33×10^{-4}	0.001366	0.001281
Type C	0.006589	1.26×10^{-4}	0.000423	0.000313

11 Debugging Gaps

If SEES fails to recover truth, debug in this order.

1. **Feature rank first.** If the reward design is not identified, larger sieves and larger penalties will only produce sharper nonsense.

2. **Basis dimension.** Low K is the standard finite-sample SEES failure mode. On this DGP, $K = 8$ recovers the policy but leaves value and parameter recovery outside the hard structural gates. The validation uses $K = 21$, equal to the finite-state value dimension.

3. **Penalty weight.** The penalty must make the Bellman residual small. A high likelihood with a loose Bellman residual is not a structural fit.

4. **Standard errors.** Non-finite Schur-complement standard errors mean the nuisance block is singular or nearly singular. Do not publish that run as a full SEES validation.

5. **Optimizer flag.** The package reports the L-BFGS-B absolute-gradient flag exactly. For SEES, the known-truth gate is the structural residual plus recovery metrics, because the sample-summed penalized objective can plateau with a large absolute gradient while the structural objects are already recovered.

6. **Scale and anchor.** If the logit scale or exit-action anchor is changed, reward levels can drift.

7. **Soft gates.** Do not accept a non-smoke run that only warns. Known-truth validation should pass hard gates or raise.

12 Practical Defaults

Recommended non-smoke validation settings:

```
SEESEstimator(
    basis_type="bspline",
    basis_dim=dgp.problem.num_states,
    penalty_weight=100.0,
    max_iter=1000,
    tol=1e-7,
    compute_se=True,
)
```

Use smoke settings only for code-path tests. A smoke test shows that the estimator runs and returns finite objects; it is not evidence of recovery.

13 Implementation Paths

- Estimator: `src/econirl/estimation/sees.py`
- Known-truth harness: `experiments/known_truth.py`
- Fast validation tests: `tests/test_known_truth.py`
- Master plan: `papers/econirl_package_jss/plans/README.md`
- SEES alignment audit: `papers/econirl_package_jss/plans/alignment/13_sees.md`

14 References

References

Yao Luo and Peijun Sang. Efficient estimation of structural models via sieves. Working Paper, University of Toronto, 2024.